Group Project Proposal

CoPrompt Adaptive Clarification Before Acting

Group ID 17

DSA4213: Natural Language Processing for Data Science

Semester 1, AY2026/2027

Administrative Information

Group members XU RUIZHE (A0354336A); ZHANG YUROU (A0353814A); XU HONGYUAN (A0357197M); PENG RUIYUN (A0356683N); LIANG CHENGWEI (A0354886J).

Project direction Adaptive clarification for task completion under a limited interaction budget.

Project Summary

We propose CoPrompt, an agent that decides whether to ask a targeted clarification question or act on the available information to produce the requested output, such as an email, short text, or study plan. We study whether an explicit question-value heuristic improves the trade-off between task quality and interaction burden compared with direct response, exhaustive clarification, and a prompted language model that decides for itself. The project uses one hosted instruction model and 120 reviewed cases across two complementary task domains: constrained writing emphasizes content and presentation constraints, while study planning emphasizes temporal and feasibility constraints. These settings expose different clarification needs without claiming broad task representativeness. The cases are split into development and test sets, and the project requires no model training. The main evidence will be constraint satisfaction and question counts on 60 held-out cases, supported by a small blinded human assessment and one bounded ablation. We aim to explain when clarification helps and when it adds effort without improving the result.

1 Problem, Research Question, and Motivation

A request such as "Help me prepare for my exam" may omit the deadline and daily study time. Acting immediately risks an unsuitable plan, while asking about every preference creates unnecessary work. Our intended users are students and other users who can describe a goal but may not know which details a language model needs. The system should manage that interaction without requiring users to write or maintain elaborate prompts.

Our research question is: Can a policy that considers the importance and uncertainty of missing information achieve a better quality-question trade-off than direct response, exhaustive clarification, and prompt-only adaptive clarification? We hypothesize that selective questioning will improve constraint satisfaction over direct response while using fewer questions than exhaustive clarification. These are hypotheses, not promised outcomes. We will report the size and uncertainty of both quality differences and question reductions; a clear account of failure conditions is also a useful project outcome.

This is an agent task because the system observes a conversation, chooses ASK or ACT, updates its state after an answer, and decides when to stop. The user can request immediate execution or decline to answer a

question.

2 Related Work and Proposed Contribution

ClariQ studies question selection in information-seeking conversations [1]. CLAM demonstrates selective clarification with language models and simulated users [2]. Zhang and Choi study when clarification is useful across NLP tasks [3]. Rao and Daumé study question ranking through the expected value of an answer [4]. These works motivate our decision problem, but our proposed score is a simple heuristic, not an estimate of calibrated expected utility.

Our contribution is a controlled empirical study of clarification policies for constrained writing and study planning. We will implement an explicit scoring and stopping controller, a simulator with defined information boundaries, a reviewed scenario set, and a common evaluation pipeline. The central comparison holds the model and generation pipeline fixed while testing the explicit controller against a prompt-only adaptive policy, alongside direct-response and exhaustive-clarification baselines. This design tests whether explicit decision logic improves the quality-question trade-off beyond asking an instruction model to decide when to clarify. We do not claim a new learning algorithm or that clarification itself is novel.

3 Agent and Task Setup

Tasks and scope. We select domains that can contain both critical missing constraints and optional preferences while allowing important output properties to be checked explicitly. Constrained writing covers short emails and notices; its fields—audience, tone, length and required content—mainly express content and presentation constraints. Study planning covers a small set of topics over a defined period; its fields—deadline, available time, topics and priorities—mainly express temporal, resource and feasibility constraints. The domains therefore provide complementary clarification settings within a manageable scope rather than representing all user tasks. Core cases contain at most three unresolved schema fields and include complete requests, important omissions, optional preferences and unavailable answers. Case authoring will cover requests with zero to three unresolved fields across these case types. Contradictory goals and changing preferences are outside the core benchmark.

Inputs and actions. The agent sees only the initial request, public field descriptions, visible answers, and the remaining budget. One shared extraction call identifies stated requirements; its result is cached for all systems. All questioning policies use the same field identifiers and short question templates. ASK requests one field; ACT uses a fixed template to assemble the initial request and visible answers for a common answer generator. Users receive the task result directly. The text interface has no web retrieval, external actions or persistent memory.

Information boundaries and stopping. Each case stores hidden requirements, predetermined field answers, availability and scoring rules. The simulator returns only the requested field answer, without LLM calls. Hidden intent, critical-field labels and scoring rules never reach the agent. An unknown or declined field is not asked again; other questions remain available. The controller stops on ACT, no eligible fields, a user request to execute, or three questions. The three-question cap matches the maximum number of unresolved fields in a core case, allows multi-step clarification while bounding trajectory length and cost, and lets the exhaustive baseline query every unresolved field under the same budget. After the third answer, the controller acts without another decision call. Repeated or invalid requests consume budget. Extraction errors are logged and never corrected using hidden annotations.

Implementation. We initially select Qwen3-30B-A3B-Instruct-2507 [5] as the shared instruction model because it is available through the course API and is expected to fit the project budget. A first-week pilot will

verify access, cost and basic task compatibility. If replacement is necessary, the alternative model will be selected before development comparisons and then frozen for all systems. All systems share the frozen model, endpoint, generation settings and answer template. Python provides the controller and runner, with a small Streamlit demonstration. Malformed outputs receive one repair attempt. If extraction still fails, we retain the original request with an empty structured state; a failed controller forces ACT. Final outputs are scored normally, and all failures are logged.

4 Proposed System or Study Design

The workflow is initial request, visible-state extraction, ASK or ACT decision, field answer and state update, then task completion. The state extractor and final answer generator are shared across systems; only the clarification policy changes. A fixed prompt template removes the need for a separate prompt-writing model.

Question value and stopping

At each decision, one LLM call rates every unresolved and available field using only the visible state and a fixed rubric. Importance is 0 for negligible effect, 0.5 for a local preference, and 1 for information likely to change the task's core content or feasibility. Uncertainty is 0.5 when a plausible default exists and 1 when materially different answers remain plausible. Resolved or unavailable fields are excluded before scoring. Scores and short reasons are logged; hidden gold labels are never used.

For field f, the controller computes $V(f) = I(f)U(f) - \lambda$, where I is importance, U is uncertainty and λ is the per-question cost threshold. The product requires a field to be both consequential and unresolved. The ratings are ordinal, not calibrated probabilities; their coding and multiplication are predefined heuristics. With the discrete scales above, $I(f)U(f)$ can take only 0, 0.25, 0.5 or 1. The controller asks the highest positive-scoring field, breaks ties by a fixed order, and reassesses after each answer. If no field has $V(f) > 0$, it acts.

Prompts and rubrics are frozen after design. Only the 20 validation cases are used to select λ from 0, 0.25 and 0.5. Because these thresholds align with the discrete $I(f)U(f)$ levels, they define progressively more conservative asking rules. Under the strict $V(f) > 0$ criterion, a field at the threshold does not trigger a question, reflecting a conservative tie-breaking rule that avoids additional interaction when the estimated question value only matches, rather than exceeds, the asking cost. We choose the setting with the best constraint satisfaction among those averaging at most two questions, breaking ties in favor of fewer questions. If none qualifies, we choose the setting with the fewest questions and report the shortfall. The two-question average is a project-defined efficiency target for selecting one operating point, not a universal user preference or a test-performance guarantee. Validation cases cannot be used to revise prompts or rubrics; λ is frozen before testing.

4.1 Comparisons and Ablations (if applicable)

We target 120 cases. The 60 development cases are divided in advance into 40 design cases for implementation, prompts and scoring rules, and 20 disjoint validation cases for threshold selection. The 12-case pilot comes from the design cases. The remaining 60 cases form the test set. Each set is balanced by domain, with scenario families separated before authoring and near-paraphrases removed. Each member drafts 24 cases for cross-review. For the blinded human subset, reviewers evaluate only cases written and cross-reviewed by other members, as specified in Section 5. No training set or synthetic expansion is required.

Each case has at least one programmatically verifiable critical constraint. All scored constraints must be jointly feasible and visible or obtainable through permitted answers. Unspecified preferences have no hidden

gold value. The three-unresolved-field limit is checked during case authoring; it is never used to correct runtime extraction with hidden annotations.

System	Clarification policy
Direct Response	Asks no questions and completes the task immediately.
Exhaustive Clarification	Asks unresolved fields in a fixed order, including optional ones, up to three questions. With correct state extraction, this covers every omission in a core case.
Prompt-only Adaptive	The same model selects ASK or ACT using a fixed instruction prompt, without explicit scoring.
Value-based Adaptive	The proposed controller selects questions and stops using importance, uncertainty and the validation-selected threshold.

The four systems run on all 60 test cases. One diagnostic uses a preselected 20-case subset: set importance to 1 for every eligible field, retaining uncertainty, threshold and budget. This tests the contribution of importance and adds 20 trajectories. There are no budget sweeps or additional models. All systems start with the same request and obtain further information only through the same simulator.

Feasibility and division of work

We plan four weeks and approximately 180 person-hours: 45 for data and scoring rules, 35 for system integration, 25 for policy development, 30 for evaluation, 30 for reporting and demonstration, and 15 for contingency. The five lead roles are research and writing; data and simulation; policy logic; baselines and integration; and evaluation and demonstration. A lead role assigns primary responsibility rather than exclusive ownership: all members contribute to case authoring, cross-review, evaluation and error analysis.

Week	Deliverable and acceptance condition
1	Freeze the schema and metrics; run the 12-case pilot. At least 10 cases must complete interaction, generation and scoring end to end. Measure calls, cost and review time.
2	Finish case review, four policies and automated checks. Freeze prompts and rubrics on the design set, then select the threshold on validation cases. Check failure recovery.
3	Freeze data and system settings; run the 60-case main test and the 20-case diagnostic. Save outputs, costs and initial result tables.
4	Complete blinded review, error analysis, report and a short demonstration. Reserve the last days for fixes and rehearsal, with no new features.

5 Preliminary Evaluation Plan

The primary quality measure is the proportion of programmatically verifiable critical constraints satisfied by the final output. We average within each case, then across cases. Checks cover word limits, explicitly required literal details such as names or reference numbers, study-time limits, deadlines and topic coverage. Plans use a common structured schedule plus readable text so dates and durations can be checked. We do not treat string matching as a measure of semantic adequacy. Failed or unparseable final outputs score zero; format failures are also reported separately.

The main interaction measure is the number of requested information items. We also log repeated or invalid requests, questions about pre-annotated optional fields, tokens, latency and actual API cost. Optional-field questions are a burden proxy, not automatically proof that the question was useless. Unsupported assumptions are assessed only in the human-review subset. These measures do not establish real-user satisfaction.

We report mean constraint satisfaction against mean question count and paired differences between Value-based Adaptive and each baseline, with 95% bootstrap intervals over matched cases or retained scenario families. Domain-level results are descriptive. All extraction, decision and generation calls use temperature = 0 and a fixed seed where the API supports it; the provider, model and remaining generation parameters are fixed and recorded. Each case has one principal run per system. Hosted inference may retain nondeterminism, so the intervals reflect case variation only. Uncertain differences remain inconclusive, not evidence of equivalent quality.

Two fixed reviewers are assigned before case authoring. From cases written and cross-reviewed by other members, we preselect 20 test cases balanced across domains and omission types. Each reviewer independently assesses all four systems and the diagnostic variant: 100 outputs and 200 assessments. System identities are hidden and order randomized. Reviewers give separate 1–5 ratings for usefulness, coherence, audience fit and feasibility, and count unsupported assumptions and unnecessary questions.

Before resolving disagreements, we compute linearly weighted Cohen's kappa per rating dimension, fixing the five-level scale in advance. Undefined kappa is reported as NA with exact agreement. Count annotations use exact agreement and mean absolute differences. Original ratings are retained. These group-member assessments apply only to the subset; we use no LLM judge and do not claim an independent user study.

We will report successful and failed trajectories and retain the reviewed data, prompts, model identifier, configurations and logs. All policy choices are frozen before testing.

6 Risks, Limitations, and Responsible Practice

The benchmark is small, uses structured fields, and simulates users. Fixed question templates make comparisons reliable but exclude free-form question generation and realistic conversational variation. The same model supplies heuristic ratings and final answers, so findings are model-specific. We will inspect poor ratings and missed information instead of presenting the score as a validated value-of-information estimator. Human reviewers are group members, and the study cannot establish real-user satisfaction or broad agent superiority.

The API allowance is USD 15, with USD 3 reserved for recovery and demonstration. No GPU rental is required. The first pilot will measure actual token use and current endpoint rates; core data and experiment plans must fit the remaining USD 12 before expansion. With three decisions per adaptive trajectory, the main test requires 240 answer-generation calls, at most 360 adaptive-decision calls and up to 60 shared extraction calls. The 20-case diagnostic adds at most 80 calls, giving about 740 test calls before repairs. Development calls and token lengths are budgeted separately from these counts. We cache identical requests, cap generation length, record expenditure and allow at most one repair per failed call.

If the pilot projects excessive cost or authoring time, we first shorten scenarios and outputs and reduce design iterations. Before validation or test runs, the fallback is 80 cases: 30 design, 10 validation and 40 test cases, each balanced by domain. The 12-case pilot remains within the design set; only the 10 validation cases select the threshold. Human review and the diagnostic share a 16-case test subset under the same reviewer exclusions, giving 160 individual assessments. We retain all four main comparisons and disclose the final sizes. Scope is never reduced in response to validation or test results. A saved trajectory backs up the demonstration.

Scenarios use synthetic personal details. Credentials and private user information will be excluded from the repository and shared logs. We will document model and data sources and software-assisted drafting as required by the course. The project involves no recruited participants; any later participant study would require separate consent and applicable institutional procedures.

References

[1] Mohammad Aliannejadi, Julia Kiseleva, Aleksandr Chuklin, Jeff Dalton, and Mikhail Burtsev. ConvAI3: Generating Clarifying Questions for Open-Domain Dialogue Systems (ClariQ). arXiv:2009.11352, 2020. https://arxiv.org/abs/2009.11352

[2] Lorenz Kuhn, Yarin Gal, and Sebastian Farquhar. CLAM: Selective Clarification for Ambiguous Questions with Generative Language Models. arXiv:2212.07769, 2022. https://arxiv.org/abs/2212.07769

[3] Michael JQ Zhang and Eunsol Choi. Clarify When Necessary: Resolving Ambiguity Through Interaction with LMs. Findings of NAACL, pp. 5541–5558, 2025. https://aclanthology.org/2025.findings-naacl.306/

[4] Sudha Rao and Hal Daumé III. Learning to Ask Good Questions: Ranking Clarification Questions using Neural Expected Value of Perfect Information. ACL, pp. 2737–2746, 2018. https://aclanthology.org/P18-1255/

[5] Qwen Team. Qwen3-30B-A3B-Instruct-2507 model card, 2025. https://huggingface.co/Qwen/Qwen3-30B-A3B-Instruct-2507